

MODUL MACHINE LEARNING



(KODE MK)

MODUL SESI 6

Algoritma Supervised Learning:

- Decision Tree (DT)
- Support Vector Machine (SVM)
- Neural Network (NN)

DISUSUN OLEH

Dr. VITRI TUNDJUNGSAARI, S.T., M.Sc, M.M.

UNIVERSITAS ESA UNGGUL 2023

INTRODUCTION

1. Kemampuan Akhir Yang Diharapkan

Setelah mempelajari modul ini, diharapkan mahasiswa mampu:

- menjelaskan konsep dasar dan aplikasi Machine Learning (ML)
- menjelaskan cara kerja algoritma ML
- mengimplementasikan data yang diproses dengan ML
- mengidentifikasi tantangan dan potensi implementasi ML

2. Uraian dan Contoh

1. Visi dan Misi

Universitas Esa Unggul mempunyai visi menjadi perguruan tinggi kelas dunia berbasis intelektualitas, kreatifitas dan kewirausahaan, yang unggul dalam mutu pengelolaan dan hasil pelaksanaan Tridarma Perguruan Tinggi.

Untuk mewujudkan visi tersebut, maka Universitas Esa Unggul menetapkan misi-misi sebagai berikut :

- a) Menyelenggarakan pendidikan tinggi yang bermutu dan relevan
- b) Menciptakan suasana akademik yang kondusif
- c) Memberikan pelayanan prima kepada seluruh pemangku kepentingan

Tujuan dari dilaksanakannya misi-misi tersebut adalah:

- 1) Sumber daya manusia yang berkarakter dan berdaya saing tinggi.
- 2) Pengembangan ilmu pengetahuan, teknologi dan seni, serta kesejahteraan umat manusia.
- 3) Perguruan tinggi yang sehat dan mandiri.
- 4) Perguruan tinggi yang bereputasi unggul.

2. Topik Perkuliahan

Pembahasan dalam matakuliah ini dimulai dengan pengenalan Machine Learning (ML) yang memiliki dua prinsip kerja, yaitu: Supervised dan Unsupervised Learning. Supervised learning merupakan pembelajaran yang memberi pelabelan terhadap data, training, dan tes sehingga dapat melakukan prediksi atau klasifikasi untuk data yang baru. Sedangkan

unsupervised learning tidak melakukan pelabelan data untuk melakukan prediksi atau klasifikasi. Pada modul ini dibahas mengenai beberapa algoritma yang digunakan dalam Supervised dan Unsupervised Learning. Selain itu, dalam modul ini juga diberikan contoh kasus untuk memberikan ilustrasi bagaimana algoritma dalam Supervised dan Unsupervised Learning bekerja. Kasus-kasus tersebut diselesaikan dengan Machine Learning menggunakan Bahasa Python. Pada bagian akhir dari modul ini dibahas tentang masalah dan tantangan yang dihadapi dalam Machine Learning.

Berikut ini topik-topik perkuliahan yang diberikan sebelum UTS:

Topik 1 – Pengenalan ML

Topik 2 – Supervised Learning vs Unsupervised Learning

Topik 3 – Algoritma Supervised Learning

Topik 4 – Algoritma Unsupervised Learning

Topik 5 -- Supervised Learning: Classification dan Regresi

Topik 6 – Supervised Learning: Decision Tree (DT)

Topik 7 – Supervised ML algorithm: Support Vector Machine (SVM)

Sedangkan topik-topik perkuliahan yang diberikan sebelum UAS adalah :

Topik 8 – Supervised Learning: Neural Network (Deep Learning)

Topik 9 – Supervised Learning: k-Nearest Neighbor

Topik 10 – Unsupervised Learning: Clustering

Topik 11 - Unsupervised Learning: Principal Component Analysis

Topik 12 – Fitur Representasi dan Rekayasa Data

Topik 13 – Evaluasi dan Perbaikan Model

Topik 14 – Tantangan dalam ML

3. Referensi dan Komponen Penilaian

Mata kuliah Machine Learning menggunakan beberapa referensi, yaitu:

1. Andreas Muller dan Sarah Guido, 2016. Introduction to Machine Learning with Python A Guide for Data Scientists: First edition.
2. Jan Wira Gotama Putra, 2020. Pengenalan Konsep dan *machine learning* dan Deep Learning, 2020: Edisi empat.

3. <https://databasetown.com/>

Untuk penilaian akhir, komponen nilai yang digunakan terdiri dari kehadiran, UTS, UAS dan penugasan. Dalam kuliah *online* komponen penugasan ditambah dengan kuis, sedangkan komponen kehadiran tidak diperhitungkan karena ditekankan pada aspek aktivitas di *website*. Adapun proporsi penilaiannya sebagai berikut :

- a. UTS = 25 %
- b. UAS = 30 %
- c. Kuis = 10 %
- d. Tugas = 35 %

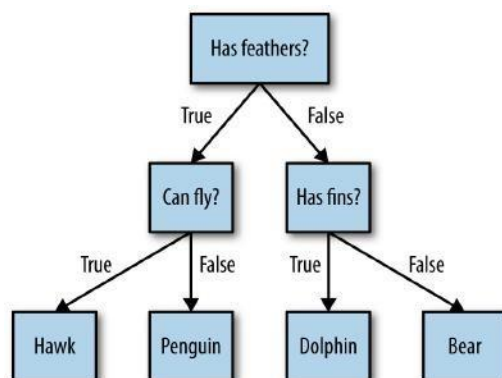
SUPERVISED LEARNING

Decision Tree

Decision tree adalah model yang banyak digunakan untuk tugas klasifikasi dan regresi. Pada dasarnya, mereka mempelajari hirarki pertanyaan jika/lainnya, yang mengarah pada sebuah keputusan.

Pertanyaan-pertanyaan ini mirip dengan pertanyaan yang mungkin Anda ajukan dalam permainan 20 Pertanyaan. Bayangkan Anda ingin membedakan antara empat hewan berikut ini: burung, elang, penguin, dan lumba-lumba. Tujuan Anda adalah untuk mendapatkan jawaban yang benar dengan mengajukan pertanyaan jika/lainnya sesedikit mungkin. Anda dapat memulai dengan menanyakan apakah hewan tersebut memiliki bulu, sebuah pertanyaan yang mempersempit kemungkinan hewan yang dapat Anda pilih menjadi hanya dua. Jika jawabannya "ya", Anda dapat mengajukan pertanyaan lain yang dapat membantu Anda membedakan antara elang dan penguin. Misalnya, Anda dapat menanyakan apakah hewan tersebut dapat terbang. Jika hewan tersebut tidak memiliki bulu, pilihan hewan yang mungkin Anda pilih adalah lumba-lumba dan burung, dan Anda perlu mengajukan pertanyaan untuk membedakan kedua hewan ini-misalnya, menanyakan apakah hewan tersebut memiliki sirip.

Rangkaian pertanyaan ini dapat dinyatakan sebagai *decision tree*, seperti yang ditunjukkan pada [Gambar 2-22](#).



Gambar 2-22. Sebuah decision tree untuk membedakan beberapa hewan

Dalam ilustrasi ini, setiap simpul dalam pohon merepresentasikan sebuah pertanyaan atau simpul akhir (disebut juga *daun*) yang berisi jawabannya. Ujung-ujungnya menghubungkan jawaban dari sebuah pertanyaan dengan pertanyaan berikutnya yang akan Anda ajukan.

Dalam istilah pembelajaran mesin, kami membangun sebuah model untuk membedakan antara empat kelas hewan (elang, penguin, lumba-lumba, dan beruang) dengan menggunakan tiga fitur "memiliki bulu", "dapat terbang", dan "memiliki sirip". Alih-alih membuat model-model ini secara manual, kita bisa mempelajarinya dari data menggunakan pembelajaran terawasi. Bagaimana cara memilih fitur dan ambang batas optimal untuk ditempatkan di setiap node?

Gini impurity

Seperti pada model pembelajaran mesin lainnya, mekanisme pelatihan pohon keputusan mencoba meminimalkan beberapa kerugian yang disebabkan oleh kesalahan prediksi pada rangkaian rangkaian kereta. Indeks *Gini impurity* (diambil dari nama ahli statistik Italia Corrado Gini) adalah ukuran alami untuk akurasi klasifikasi.

$$Gini = 1 - \sum_{i=1}^n p^2(c_i)$$

$$Entropy = \sum_{i=1}^n -p(c_i)\log_2(p(c_i))$$

Gbr.4 — Dua tujuan rangkaian umum untuk pohon: Gini dan entropi. $P(c_i)$ adalah probabilitas pengambilan sampel kelas c_i secara acak dari populasi. n adalah jumlah kelas.

Gini yang tinggi menunjukkan populasi yang heterogen (jumlah sampel yang sama dari setiap kelas) sedangkan Gini yang rendah menunjukkan populasi yang homogen (yakni sebagian besar terdiri dari satu kelas)

Nilai Gini maksimum yang mungkin bergantung pada jumlah kelas: dalam masalah klasifikasi dengan kelas C , Gini maksimum yang mungkin adalah $1-1/C$ (bila kelas-kelas

tersebut terisi secara merata). Gini minimum adalah 0 dan dicapai bila seluruh populasi terdiri dari satu kelas.

Indeks *Gini impurity* merupakan nilai ekspektasi kesalahan klasifikasi jika klasifikasi dilakukan secara acak. Karena peluang terambilnya sampel secara acak dari kelas c_i adalah $p(c_i)$. Setelah dipilih, peluang memprediksi kelas yang salah adalah $(1-p(c_i))$. Jika kita menjumlahkan $p(c_i)*(1-p(c_i))$ pada semua kelas, kita mendapatkan rumus *Gini impurity* pada Gambar 4.

Dengan indeks Gini sebagai tujuan, pohon pada setiap langkah memilih fitur dan ambang batas yang membagi populasi sedemikian rupa sehingga menurunkan Gini rata-rata tertimbang dari dua populasi yang dihasilkan secara maksimal (atau meningkatkan homogenitas rata-rata tertimbang secara maksimal). Dengan kata lain, logika kereta api adalah meminimalkan kemungkinan kesalahan klasifikasi acak pada dua populasi yang dihasilkan, dengan bobot yang lebih besar diberikan pada sub populasi yang lebih besar.

Dari definisi ini kita juga dapat memahami mengapa nilai ambang batas selalu merupakan nilai aktual yang ditemukan pada setidaknya satu sampel kereta - tidak ada gunanya menggunakan nilai yang ada dalam kesenjangan antar sampel karena pemisahan yang dihasilkan akan sama.

Metrik lain yang biasa digunakan untuk pelatihan pohon adalah entropi (lihat rumus pada Gambar.4).

Entropi

Sementara strategi Gini bertujuan untuk meminimalkan kesalahan klasifikasi acak pada langkah berikutnya, strategi minimalisasi entropi bertujuan untuk memaksimalkan perolehan informasi.

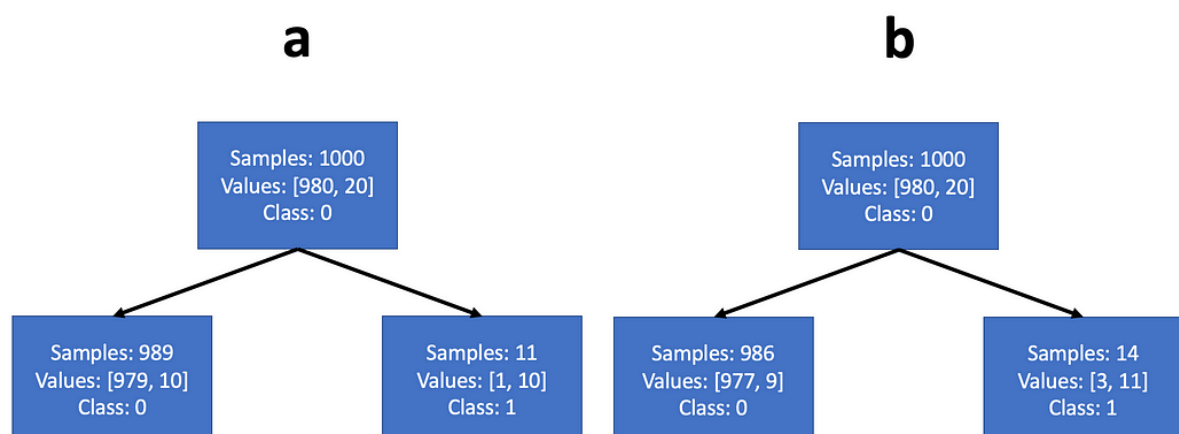
Seperti Gini, meminimalkan entropi juga sejalan dengan menciptakan populasi yang lebih homogen, karena populasi homogen memiliki entropi yang lebih rendah (dengan populasi kelas tunggal yang ekstrim memiliki entropi 0 — tidak perlu menanyakan pertanyaan ya/tidak).

Gini atau Entropi?

Sebagian besar sumber mengklaim bahwa perbedaan antara kedua strategi tersebut tidak terlalu signifikan (memang — jika Anda mencoba melatih pohon entropi pada masalah yang baru saja kita kerjakan — Anda akan mendapatkan pembagian yang persis sama). Alasannya mudah untuk diketahui: meskipun Gini memaksimalkan nilai ekspektasi probabilitas kelas, entropi memaksimalkan nilai ekspektasi probabilitas kelas log. Namun probabilitas log adalah fungsi probabilitas yang meningkat secara monoton, sehingga biasanya cara kerjanya hampir sama. Namun, minimalisasi entropi mungkin memerlukan konfigurasi yang berbeda dari Gini, ketika populasi sangat tidak seimbang. Misalnya, kita dapat membayangkan kumpulan data dengan 1000 sampel pelatihan - 980 milik kelas 0 dan 20 milik kelas 1. Katakanlah pohon dapat memilih ambang batas yang akan membaginya berdasarkan contoh a atau contoh b pada Gambar 5. Kita perhatikan bahwa kedua contoh tersebut menciptakan sebuah simpul dengan populasi besar yang sebagian besar terdiri dari kelas mayoritas, dan simpul kedua dengan populasi kecil yang sebagian besar terdiri dari kelas minoritas.

Dalam kasus seperti ini, kecuraman fungsi log pada nilai yang kecil akan memotivasi kriteria entropi untuk memurnikan node dengan populasi yang besar, lebih kuat dari kriteria Gini. Jadi jika kita menghitungnya, kita akan melihat bahwa kriteria Gini akan memilih pemisahan a, dan kriteria entropi akan memilih pemisahan b.

Hal ini dapat menyebabkan pilihan fitur/ambang batas yang berbeda. Belum tentu lebih baik atau lebih buruk — ini tergantung pada tujuan kita.



Akhir pelatihan

Ketika jalur di pohon mencapai nilai kedalaman yang ditentukan, atau ketika jalur tersebut berisi populasi Gini/entropi nol, maka pelatihan akan dihentikan. Ketika semua jalur berhenti dilatih, pohon sudah siap.

Praktik yang umum dilakukan adalah membatasi kedalaman pohon. Cara lainnya adalah dengan membatasi jumlah sampel dalam satu daun (tidak memperbolehkan sampel lebih sedikit dari ambang batas). Kedua praktik tersebut dilakukan untuk mencegah overfitting pada *train set*.

Keuntungan:

1. Dibandingkan dengan algoritma lain, pohon keputusan memerlukan lebih sedikit usaha untuk persiapan data selama pra-pemrosesan.
2. Pohon keputusan tidak memerlukan normalisasi data.
3. Pohon keputusan juga tidak memerlukan penskalaan data.
4. Nilai yang hilang dalam data juga TIDAK mempengaruhi proses pembuatan pohon keputusan sampai batas tertentu.
5. Model pohon keputusan sangat intuitif dan mudah dijelaskan kepada tim teknis serta pemangku kepentingan.

Kerugian:

1. Perubahan kecil pada data dapat menyebabkan perubahan besar pada struktur pohon keputusan sehingga menyebabkan ketidakstabilan.
2. Untuk pohon Keputusan terkadang penghitungan bisa menjadi jauh lebih kompleks dibandingkan dengan algoritma lainnya.
3. Pohon keputusan sering kali memerlukan waktu yang lebih lama untuk melatih model.
4. Pelatihan pohon keputusan relatif mahal karena kompleksitas dan waktu yang dibutuhkan lebih banyak.
5. Algoritma Pohon Keputusan tidak memadai untuk menerapkan regresi dan memprediksi nilai kontinu.